

What is *ZEUS-3D*?

David A. Clarke
Professor (retired) of astronomy and physics
Halifax NS, Canada
zeus3d.ca/zeus3d/version3.5
Copyright © David A. Clarke, 2026

Contents

1. Introduction 1
2. Physics 2
3. Features and Algorithms 3
3.1 Overview and critiques 3
3.2 Geometry 5
3.3 MHD algorithms 6
3.4 Boundary conditions 8
3.5 Self-gravity algorithms 9
3.6 Interpolation schemes 9
3.7 Moving grid 10
3.8 Graphics and I/O 10
3.9 Parallelism, efficiency and memory occupancy 10
References 11

1 Introduction

ZEUS-3D is a multi-physics computational fluid dynamics (CFD) code written in FORTRAN designed primarily for, but not restricted to, astrophysical applications. It was born from the *ZEUS*-development project headed by Michael Norman at the NCSA in the late 1980s and early 1990s whose principal developers included myself (1986–1988; `zeus04`, 1990–1992; *ZEUS-3D*), Jim Stone (1988–1990; *ZEUS-2D*), and Robert Fiedler (1992–1994; `ZEUSMP`). Each of *ZEUS-2D* and *ZEUS-3D* (version 3.2; `zeus32`) were placed in the public domain in 1992, followed two years later by `ZEUSMP`, an *MPI* version of `zeus32`. These codes are still available from the *Laboratory for Computational Astrophysics* (LCA) site, now hosted by the University of California (San Diego).

The NCSA/UCSD/LCA version of the code has undergone relatively little algorithm development since the release of `ZEUSMP`, and the evolution of the code has been driven by the multitude of investigators who have modified it for their own purpose. This author has continued to develop *ZEUS* (Clarke, 1996; 2010), making it available to collaborators and others upon request but not from a publicly-accessible site. Thus, this (`dzeus35`; double

precision, version 3.5) is the first major re-release of *ZEUS-3D* into the public domain by one of the original authors in more than a decade. It is a completely different code than version 3.2, though still recognisable as a member of the *ZEUS*-family of codes by its adherence to a staggered grid.

This document gives an overview of some of the capabilities of the code, the algorithms used, and addresses some of the criticisms the code has received since its first release. This document is not a "how-to" manual for the code; for that, the reader is directed to `dzeus35_man.ps` in the `documents` directory of the file `dzeus35.tar` downloadable from this site. Instead, this document introduces the user to the functionality of the *ZEUS-3D* code, and addresses some of the philosophical and technical issues of its design.

2 Physics

As a multi-physics and covariant CFD code, *ZEUS-3D* includes the effects of: magnetism; self gravity (with an "N–log N" Poisson solver); Navier-Stokes viscosity; a second cospatial and diffusive fluid; molecular cooling; and isothermal, adiabatic, or polytropic equations of state. Terms in equations (1)–(6) below are colour-coded respectively; black terms being the standard equations of ideal HD). Simulations may be performed in any of Cartesian (`XYZ`), cylindrical (`ZRP`), or spherical polar coordinates (`RTP`), and in any dimensionality (1-D, $1\frac{1}{2}$-D, 2-D, $2\frac{1}{2}$-D, or 3-D). The set of equations solved are:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho\, \vec{v}) = 0; \tag{1}$$

$$\frac{\partial \vec{s}}{\partial t} + \nabla \cdot \left(\vec{s}\vec{v} + (p_1 + p_2 + p_B)\, \mathsf{I} - \vec{B}\vec{B} - \mu\, \mathsf{S} \right) = -\rho\, \nabla\phi; \tag{2}$$

$$\frac{\partial e_1}{\partial t} + \nabla \cdot (e_1 \vec{v}) = -p_1 \nabla \cdot \vec{v} + \mu \mathsf{S} : \nabla \vec{v} - \mathcal{L}; \tag{3}$$

$$\frac{\partial e_2}{\partial t} + \nabla \cdot (e_2 \vec{v} - \mathsf{D} \cdot \nabla e_2) = -p_2 \nabla \cdot \vec{v}; \tag{4}$$

$$\frac{\partial e_T}{\partial t} + \nabla \cdot \Big[(e_T + p_1 + p_2 - p_B)\, \vec{v} - \mu \mathsf{S} \cdot \vec{v} - \mathsf{D} \cdot \nabla e_2 + \vec{E} \times \vec{B} \Big] = -\mathcal{L}; \tag{5}$$

$$\frac{\partial \vec{B}}{\partial t} + \nabla \times \vec{E} = 0, \tag{6}$$

where:

ρ	is the matter density;
$\vec{v}$	is the velocity;
$\vec{s}$	is the momentum density $= \rho\vec{v}$;
p_1 & p_2	are the partial pressures from the first and second fluids;
p_B	is the magnetic pressure $= \frac{1}{2}B^2$;
I	is the unit tensor;
$\vec{B}$	is the magnetic induction (in units where $\mu_0 = 1$);

μ is the shear viscosity;

S is the viscid part of the stress tensor whose elements, S_{ij}, are given by:

$$S_{ij} = \partial_j v_i + \partial_i v_j - \tfrac{2}{3}\delta_{ij}\nabla\cdot\vec{v};$$

where ∂_i indicates partial differentiation with respect to the coordinates, x_i, $i = 1, 2, 3$, and where δ_{ij} is the usual "Kronecker delta";

ϕ is the gravitational potential, $\nabla^2\phi = 4\pi\rho$, in units where $G = 1$;

e_1 & e_2 are the internal energy densities of the first and second fluids;

$\mathcal{L}$ is the cooling function (of ρ and e_1) for nine coolants (HI, HII, CI, CII, CIII, OI, OII, OIII, SII) interpolated from tables given by Raga *et al.* (1997);

D is the (diagonal) diffusion tensor;

e_T is the total energy density $= e_1 + e_2 + \frac{1}{2}\rho v^2 + \frac{1}{2}B^2 + \phi$;

$\vec{E}$ is the induced electric field $= -\vec{v}\times\vec{B}$;

and where the ideal equations of state close the system of variables:

$$p_1 = (\gamma_1 - 1)e_1; \qquad p_2 = (\gamma_2 - 1)e_2,$$

with γ_1 and γ_2 being the ratios of specific heats for the two fluids.

Radiation MHD using the flux-limited diffusion approximation (*e.g.*, Turner & Stone, 2001) is only partially implemented and thus not included above. The Raga cooling functions are available upon request in a separate *change deck*[1], but are not fully debugged. Note that one uses *either* the internal energy equation, (3), *or* the total energy equation, (5), depending upon whether a strictly positive-definite internal energy density, e_1, or strict conservation of energy, e_T, is paramount.

3 Features and Algorithms

3.1 Overview and critiques

Version 3.5 of *ZEUS-3D* uses numerous different algorithms and, in some cases, different choices of algorithm for the same capability are supported. In this release, the code can be configured as fully conservative for 1-D simulations (by specifying the total energy equation) but, in multi-dimensions, the momentum equation is not fully conservative in its treatment of the transverse Lorentz forces. The momentum and internal energy equations are operator split, meaning the source terms [*i.e.*, pressure gradient, Lorentz force, rotational pseudo-forces and gravity in equation (2); adiabatic cooling and heating—the $p\nabla\cdot\vec{v}$ term—in equations (3) and (4)] are advanced before the transport steps (respectively the $\nabla\cdot\vec{s}\vec{v}$ and $\nabla\cdot e\vec{v}$ terms) are performed. In the next release of the code, the momentum equation will

[1] A change deck is a text file containing only the changes to the `FORTRAN` plus logic read by a precompiler indicating where the changes go in the code. See `dzeus35.man.ps` in the directory `documents` in the file `dzeus35.tar` downloadable from this site for full operational details.

be fully conservative and unsplit as this is needed for the *AMR* version of the code, AZEuS, currently under development. By its very nature, the internal energy equations will remain non-conservative and operator split, and the user will be able to select the conservative and unsplit total energy equation if a purely conservative and unsplit code is desired.

To be clear, *ZEUS-3D* is *not* a *Godunov*-type code. Instead, *ZEUS-3D* employs a *staggered mesh* in which the scalars (ρ, e_1, e_2, e_T) are zone-centred and components of the principal vectors ($\vec{v}$, $\vec{B}$) are face-centred (Clarke, 1996). Secondary vectors such as the current density ($\vec{J}$) and the induced electric field ($\vec{E}$) are edge-centred. Thus, fluxes for zone-centred quantities are computed directly from the time-centred velocities at zone interfaces; no interpolations need be performed and no characteristic equations need be solved to obtain these velocities. In many cases, this allows shocks and contacts to be captured in as few zones as the so-called Godunov-type "shock-capturing" schemes (*e.g.*, Dai & Woodward, 1994; Ryu & Jones, 1995; Balsara, 1998; Falle *et al.*, 1998) in which all variables are zone-centred and fluxes are computed from the face-centred primitive variables determined by the (approximate) solution of the Riemann problem. See problem #16 in the *ZEUS-3D* 1-D *Gallery* on this web site (`www.ica.smu.ca/zeus3d`) for further discussion on this point.

Besides simplicity in coding, the principal advantage of the staggered mesh is the *solenoidal condition* on the magnetic field ($\nabla \cdot \vec{B} = 0$) is maintained trivially everywhere on a 3-D grid to within machine round-off error. Zone-centred schemes typically require a "flux-cleansing" step (*e.g.*, Dai & Woodward, 1994) or a diffusion step (*e.g.*, *FLASH*; Fryxell *et al.*, 2000) to eliminate or reduce numerically driven magnetic monopoles after each MHD cycle. "Hybrid" Godunov schemes with two sets of magnetic field components—one face-centred, one zone-centred—have also been developed (Balsara & Spicer, 1999; Gardiner & Stone, 2005) which require an additional round of two-point averaging and/or interpolation between the zone-centred and face-centred magnetic field components (or, more accurately, the fluxes and induced fields) in each MHD cycle. While flux-cleaning and hybrid methods both eliminate magnetic monopoles to machine round-off error, they introduce diffusion into the scheme whose effects, if any, have not been fully investigated. The diffusion step used in *FLASH* spreads monopoles around the grid rather than eliminating them, and this can pose problems in the vicinity of shocks where monopoles can build up faster than the diffusion step can remove them (Dursi, 2008, *private communication*).

ZEUS-3D is *upwinded* in the flow and Alfvén waves and *stabilised* on compressional waves (sound waves in HD, fast and slow magnetosonic waves in MHD). Stabilisation is accomplished via a von Neumann-Richtmyer artificial viscosity (von Neumann & Richtmyer, 1950; see also Stone & Norman, 1992a; Clarke, 1996; 2010). Most Godunov-type methods also require some artificial viscosity to maintain stability though typically a factor of two or so less. As shock waves are self-steepening, shocks are typically captured by *ZEUS-3D* in two or three zones, although slow-moving shocks may typically be spread out over several more. Both contact discontinuities and Alfvén waves are diffused as they move across a grid, and will gradually spread over many zones. However, the code includes a contact steepener (Colella & Woodward, 1984) which, when engaged, introduces a local (to the discontinuity) "anti-diffusive" term that can maintain a contact within one or two zones. However and on occasion, the steepening algorithm can introduce small oscillations in the flow. To illustrate these points, problems #1–12 in the *ZEUS-3D* 1-D Gallery give the `dzeus35` solutions to

the full suite of 1-D Riemann problems studied by Ryu & Jones (1995).

Falle (2002) points out that *ZEUS* will, on occasion, attempt to fit a rarefaction wave with so-called "rarefaction shocks" (problems #13–14 in the *ZEUS-3D* 1-D *Gallery*), and attributes this to the operator-split nature of the algorithm to solve the momentum equation (and specifically, because this makes the code first-order in time, but second order in space). Applying sufficient artificial viscosity (and for most cases, a very modest amount will do) will normally correct this tendency by rendering the local spatial accuracy to first order too. If Falle is correct, the non-operator split scheme for the momentum equation currently being developed should also address this issue.

Falle (2002) also gives two examples of 1-D Riemann-type problems in which *ZEUS* converges to "incorrect values", and from this concludes "it is not satisfactory for adiabatic MHD". In fact, the failing is with how *ZEUS* was used, and not with the underlying MHD algorithm. Falle used *ZEUS* in its "non-conservative form" [*i.e.*, using the internal energy equation (3)] and this does indeed cause some variables, often the density and pressure, to converge to different levels than with a conservative code (problems #15–16 in the *ZEUS-3D* 1-D *Gallery*). In fact, since the analytic solution to the Riemann problem is normally obtained from the *conservative* equations, it shouldn't come as any surprise that the *non*-conservative equations arrive at different solutions in situations where the loss of energy is important. Indeed, *ZEUS-3D* in its non-conservative form arrives at *precisely the correct levels* to the non-conservative Riemann problem (with an appropriate model for energy loss), and comparisons with the conservative solutions are inappropriate. On the other hand, in its conservative form (*e.g.*, Clarke, 1996) `dzeus35` does, in fact, converge to the analytic solution of the conservative Riemann problem providing, for the most part, numerical solutions just as "crisp" as fully upwinded schemes such as TVD (Ryu & Jones, 1995) or that developed and used by Falle (2002); see discussion in problem #16 of the *ZEUS-3D* 1-D Gallery. For a full response to the criticisms of the *ZEUS* algorithms levelled by Falle and others, see Clarke (2010).

For multi-dimensional calculations, the code can be run in a completely directionally split fashion, or in a hybrid scheme in which compressional/longitudinal terms (*e.g.*, $\partial_i \rho$, $\partial_i v_i$, $\partial_i B^2$, *etc.*) are directionally split, while transverse/orthogonal terms (*e.g.*, $\partial_i v_j$, $B_i \partial_i B_j$, $\partial_i v_i B_j$, $i \neq j$, *etc.*) are *planar*-split (Clarke, 1996). This is described more fully in §3.3.

3.2 Geometry

The code is written in a "covariant" fashion, meaning key vector identities (*e.g.*, $\nabla \cdot \nabla \times \vec{A} = 0$; $\nabla \times \nabla \psi = 0$) are preserved to machine round-off error for the three most commonly used orthogonal coordinate systems: Cartesian (`XYZ`), cylindrical (`ZRP`), and spherical polar (`RTP`). The user should be aware that the conservative nature of the momentum equation (available in 1-D only in this release) depends upon the use of Cartesian coordinates where the gradient and divergence derivatives are identical. In curvilinear coordinates where the metric factors render the gradient and divergence derivatives distinct, so-called "pseudo-forces" in the momentum equation (and analogous magnetic terms from $\vec{J} \times \vec{B}$) introduce imperfect derivatives which render the algorithm non-conservative. This may have effects on some specialised and pedagogical 1-D and 2-D test problems but, in practise, should have little if any measurable consequence on a fully dynamic simulation.

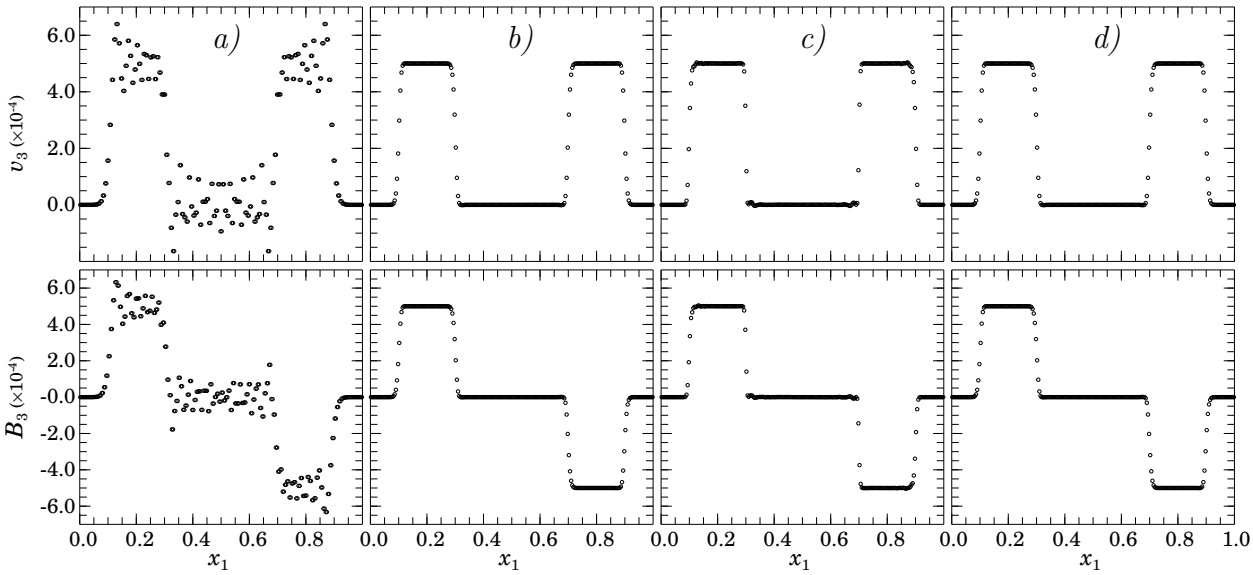


Figure 1: A square wave perturbation (10^{-3}) is applied to v_3 in the central 50 zones of a 1-D system resolved with 250 zones where $\rho = 1$, $p = 0.6$, $v_1 = 0$, and $B_1 = 1$, and allowed to propagate to $t = 0.75$. From left to right are the results for *a)* no upwinding in the Alfvén characteristics; *b) MoC*; *c) HSMoC*; and *d) CMoC*. Further discussion is in the text.

In addition, the code is designed to run as efficiently in 1-D or 2-D as a specifically-written 1-D or 2-D code. That is to say, if the user wishes to run the code in its 2-D mode, one doesn't have to worry that the actual execution would be a 3-D code with a few zones in the symmetry direction. In fact, in its 2-D mode, *ZEUS-3D* will only compute the fluxes in the two active directions, and arrays will be declared completely as 2-D entities.

3.3 MHD Algorithms

Three MHD algorithms are included in this release. The Stone & Norman *Method of Characteristics* (*MoC*; Stone & Norman, 1992b) was invented so that MHD calculations could be upwinded in the Alfvén speed, a critical attribute for any stable MHD algorithm. Figures 1*a* and 1*b* show respectively the results after a 1-D Alfvén wave has propagated for 1.5 pulse-widths in either direction without *MoC* and with *MoC*.

Shortcomings with *MoC* become apparent in multi-dimensions. Clarke (1996) showed that *MoC* is subject to "magnetic explosions", a numerical instability most evident in super-Alfvénic turbulence in which local kinetic and magnetic energy densities can interchange, thereby suddenly and catastrophically introducing a dynamically important magnetic field into an otherwise high-β flow inside a single MHD cycle. Clarke (1996) also showed that while this is a local effect, it can have serious global consequences. Integrated across the entire grid, *MoC* will cause the magnetic energy density to grow exponentially with e-folding times much shorter than can be explained physically by turbulence (*e.g.*, Clarke, 2010).

Hawley & Stone (1995) devised the so-called *HSMoC* in which the original *CT* scheme (Evans & Hawley, 1988, and used to generate Fig. 1*a*) is averaged with *MoC*. Figure 1*c* shows that an Alfvén wave transported by *HSMoC* is stable but, as implemented in this

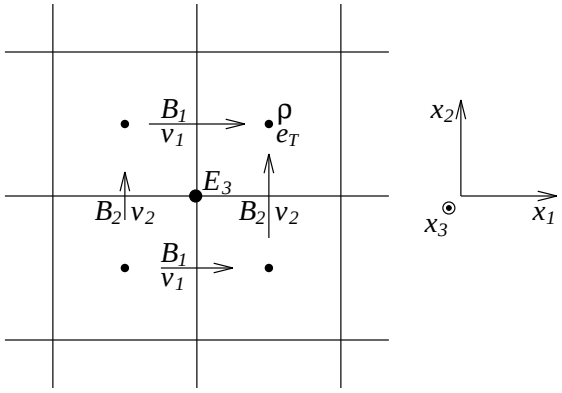


Figure 2: A segment of a Cartesian grid centred over the 3-edge where the 3-component of the induced electric field (E_3) is located. Shown also are two locations each for the 1- and 2-components of the velocity and magnetic field which must be interpolated to the 3-edge in order to make an estimate of E_3. In this projection, v_3 and B_3 (not shown) would appear co-spatial with the zone-centred quantities, ρ and e_T. In fact, they are staggered back and forward half a zone in the 3-direction.

release, requires twice as many time steps as *MoC* to maintain that stability (the "Courant number" has to be 0.5 or less). Still, Hawley & Stone (1995) claim *HSMoC* is more resilient to the "explosive" field instability than straight *MoC*, although it is unclear why this should be given that the hybrid scheme does not address the fundamental cause of the instability (described below).

The *Consistent Method of Characteristics* (*CMoC*; Clarke 1996) introduces a paradigm shift in how the induction equation is treated numerically. The basic principle of *MoC* remains intact: transverse interpolations of $\vec{v}$ and $\vec{B}$ are upwinded in the Alfvén characteristics. However, rather than doing this in a *directionally-split* fashion, *CMoC* is *planar*-split and this completely cures the magnetic explosion instability in super-Alfvénic turbulence.

The issue is this. In order to estimate the 3-component of the induced electric field ($E_3 = v_1 B_2 - v_2 B_1$ located at the 3-edge), upwinded 1-interpolations of v_2 and B_2 and upwinded 2-interpolations of v_1 and B_1—all to the 3-edge—are needed simultaneously (see Fig. 2). However, before the 1-interpolated 2-components can be computed, 2-interpolated 1-components are needed to construct the 1-characteristics while, at the same time, before the 2-interpolated 1-components can be computed, 1-interpolated 2-components are needed to construct the 2-characteristics. To break this apparent "catch-22", the directionally-split *MoC* makes preliminary estimates of all vector components at the 3-edges by taking two-point averages and using these to construct the required characteristics. The problem is, the two-point averages and subsequent upwinded values represent two independent estimates of the same quantities at the same location and time and these can be radically different particularly at shear layers. These differences are at the root of the explosive magnetic field instability in *MoC* and have to be eliminated to cure the problem.

To this end, the planar-split *CMoC* considers the entire 1-2 plane as one entity and identifies the upwinded *quadrant* relative to the 3-edge (rather than the upwinded *direction* as in the directionally-split *MoC*) thereby finding the 1- and 2-interpolations simultaneously.

The rather elaborate algorithm to perform these implicit interpolations is described in detail in Clarke (1996), and rendered in FORTRAN in the `CMOC*` routines in `dzeus35`. As a result, the 1-components used to construct the 1-characteristics for the 1-interpolated 2-components are *consistent* with (the same as) the 2-interpolated 1-components, and *vice versa*, whence the leading '*C*' in *CMoC*. Figure 1*d* shows the 1-D Alfvén wave transported by *CMoC* and, as *CMoC* reduces to *MoC* in 1-D, it is no surprise that this plot is identical to Fig. 1*b*.

The original *MoC* as described by Stone & Norman (1992b) and embodied in the NCSA releases of the code (`zeus32` and `zeusmp`) consists of a rather complicated hybrid of a *Lagrangian step* for the transverse Lorentz forces followed by an *Eulerian* step for the induced fields and then a separate transverse momentum update whose fluxes are constructed from values upwinded in the flow speeds only. In fact, a much simpler scheme is possible involving a single Eulerian step whose interpolated values are used for all three purposes (transverse Lorentz forces, induced electric field, *and* transverse momentum transport), and this is how each of *MoC*, *HSMoC*, and *CMoC* are implemented in this release of *ZEUS-3D* (`dzeus35`).

Finally, the planar-split *CMoC* has been criticised by some as being *overly* complex, and indeed it is much more complicated to program than the *MoC*, for example. Still, the complexity of planar splitting over directional splitting is no more than the complexity of the curl (the operator in the induction equation) over the divergence (the operator in the hydrodynamical equations where directional splitting is appropriate), and thus planar splitting is no more complex than it needs to be. Of note is the fact that *CMoC* in `dzeus35` has been used for more than a decade for a wide number of applications with no indication of numerical instability or "sensitivity" in all dimensionalities.

3.4 Boundary conditions

A variety of MHD boundary conditions are supported. Boundaries are set independently for each of the six (in 3-D) boundaries and, within each boundary, the boundary *type* may be set zone by zone, if needed. The ten boundary types currently available are:

1. reflecting boundary conditions at a symmetry axis ($r = 0$ in `ZRP`, $\theta = 0$ or π in `RTP`) or grid singularity ($r = 0$ in `RTP`);
2. *non-conducting* reflecting boundary conditions (Fig. 3*a*);
3. *conducting* reflecting boundary conditions (Fig. 3*b*);
4. *continuous* reflecting boundary conditions (Fig. 3*c*);
5. periodic boundary conditions;
6. "self-computing" boundary conditions (for *AMR*);
7. transparent (characteristic-based) outflow boundary conditions (not implemented);
8. "selective" inflow boundary conditions (useful for sub-fast-magnetosonic inflow);
9. traditional outflow boundary conditions (variables constant across boundary);

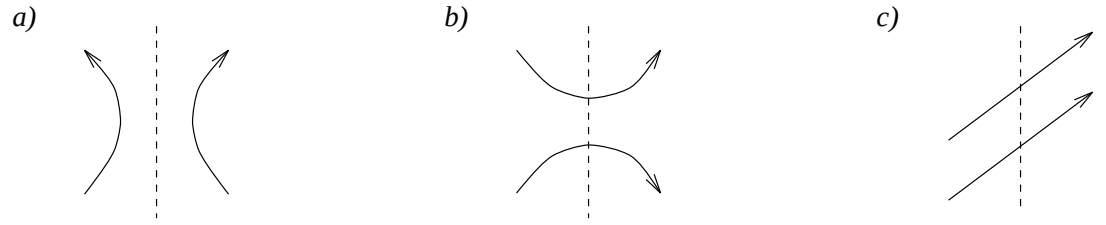


Figure 3: Panels show magnetic field lines across three different types of reflecting boundaries. *a)* non-conducting reflecting boundary conditions impose $B_{\parallel}(\text{out}) = -B_{\parallel}(\text{in})$, $B_{\perp}(\text{out}) = B_{\perp}(\text{in})$; *b)* conducting reflecting boundary conditions impose $B_{\parallel}(\text{out}) = B_{\parallel}(\text{in})$, $B_{\perp}(\text{out}) = -B_{\perp}(\text{in})$; and *c)* continuous reflecting boundary conditions impose $B_{\parallel}(\text{out}) = B_{\parallel}(\text{in})$, $B_{\perp}(\text{out}) = B_{\perp}(\text{in})$. The designations $\parallel$ and $\perp$ are relative to the boundary *normal*.

10. traditional inflow boundary conditions (suitable for super-fast-magnetosonic inflow).

All boundary conditions strictly adhere to the magnetic solenoidal condition, even where different boundary types may adjoin on the same boundary. The first five conditions may be considered *exact*, while the latter five are approximate and can launch undesired waves into the grid under some circumstances.

3.5 Self-gravity algorithms

Two algorithms for solving Poisson's equation are included in *ZEUS-3D*: *Successive Over-Relaxation* (*SOR*), and *Full Multi-Grid* (*FMG*). Both algorithms were taken from Press *et al.*(1992). *SOR* is an "N^2" algorithm and, while simple to program and understand, will dominate the computing time required for grids larger than 32^3 (*i.e.*, take longer than the MHD update). *FMG* is an example of an "N-log N" algorithm and, while rather complex, requires a comparable amount of computing time to the MHD cycle regardless of grid size. There is no measurable difference between the "accuracies" of *SOR* and *FMG*, and the only "nuisance" about *FMG* is that it requires the number of zones in each direction to be one less than a power of two.

Boundary conditions for the gravitational potential are different from the MHD conditions described in §3.4. Only two methods for specifying gravitational boundary conditions are supported. Analytically determined boundaries may be set, or boundaries may be set by performing a multipole expansion (to include the *dotridecapole*) to the boundary. The latter requires that the vast majority of matter be contained well inside the bounded region.

Gravitational periodic boundaries are not available in this release.

3.6 Interpolation schemes

Numerous interpolation schemes (for determining fluxes) are available in *ZEUS-3D*. Scalars may be interpolated with a first-order donor cell, one of three second order "van Leer" schemes (van Leer, 1977) including a "velocity corrected scheme which takes into account the fact that the face-centred velocity can vary across the zone, and Colella & Woodward's

(1984) third order *Piecewise Parabolic Interpolation* (*PPI*) scheme including their contact steepener. In *ZEUS-3D*, *PPI* has been modified with a *global extremum detector* which prevents global extrema from being monotonised (clipped) making it better suited for regions of smooth flow. This feature does not interfere with *PPI*'s renowned ability to maintain numerical monotonicity and to detect and steepen discontinuities. Vector components may be interpolated in the longitudinal direction using donor cell, van Leer interpolation, or *PPI*, while transverse interpolations (using the implicit bi-directional method described in §3.3) may be donor cell or van Leer only.

3.7 Moving grid

`dzeus35` has inherited the "moving-grid" infrastructure designed by Jim Wilson and Michael Norman in the 1970s. It is a pseudo-Lagrangian technique in which a grid line can be given the average speed of flow across it, thereby reducing grid diffusion and increasing the time step. It is also useful in simulations such as a blast front, where the structures may be followed for great lengths of time without having to define an enormously long grid through which the front moves. While an attempt has been made to carry this feature with all the new developments over the past two decades, it has never been used or tested by this author. Therefore, any user wishing to use this feature is cautioned that some significant debugging and code development may be needed in order to restore it to its full capability. If anyone needing this utility elects to undertake this task, their change deck implementing their modifications into the code would be greatly appreciated by the developers so that it may be included—with due credit—in a future release of the code.

3.8 Graphics and I/O

Numerous I/O options are supported *in-line*, including 1-D and 2-D graphics (*NCAR* and *PSPLOT*), 2-D pixel dumps (for animations), a full-accuracy dump for check-pointing, *HDF* dumps (version 4.2; useful for *IDL* and other commercially available visualisation packages), and line-of-sight integrations for a variety of flow and emission variables (*RADIO*).

3.9 Parallelism, efficiency, and memory occupancy

`dzeus35` is parallelisable for an SMP (shared memory platform) environment and the necessary *OpenMP* commands (and *FPP* commands under Cray's `UNICOS`) are automatically inserted into all routines, including those introduced by the user, by the precompiler *EDITOR* which comes with the download. The code is *not* written with *MPI* commands and thus is not suitable for a DMP (distributed memory platform) such as a "Beowulf" cluster. Parallel speed-up factors depend a great deal on the platform and how the grids are declared. On our 16-way SMP (2 GHz AMD Opteron dual core) and for a 60^3 grid, we find the code to be about 98% parallelised giving us a speed-up factor of about 12.5.

Not including I/O, `dzeus35` uses about 1,500, 2,200, and 3,000 floating point operations ("flops") per MHD zone update (no self-gravity) for 1-D, 2-D and 3-D simulations respectively. These include all adds, multiplies, divides, if tests, subroutine calls, *etc.* required to update a single zone. For pure hydrodynamics, expect about half these number of flops; for

self-gravitating MHD (using FMG), double. Thus, for example, a 256^3 3-D MHD simulation evolved for 25,000 time steps requires about $3 \times 10^3 \times 1.7 \times 10^7 \times 2.5 \times 10^4 \sim 10^{15}$ flops to complete. On a single 2 GHz core with 50% efficiency, this translates to $\sim 10^6$ s, or about 350 cpu hours. On a dedicated 16-way SMP with a speed-up factor of 12.5, the same simulation takes about 30 wall-clock hours.

Finally, the memory occupancy for a 3-D MHD (HD) calculation is equivalent to about 20 (16) 3-D arrays. For a 256^3 calculation, this means about $20 \times 1.7 \times 10^7 \times 8$ bytes (for double precision) ~ 3 GBytes.

References

Balsara, D. S., 1998, ApJS, 116, 133.
Balsara, D. S. & Spicer, D. S., 1999, J. Comput. Phys., 149, 270.
Clarke, D. A., 1996, ApJ, 457, 291.
——., 2010, ApJS, 187, 119.
Colella, P., & Woodward, P. R., 1984, J. Comput. Phys., 54, 174.
Dai, W., & Woodward, P. R., 1994, J. Comput. Phys., 111, 354.
Evans, C. R., Hawley, J. F., 1988, ApJ, 332, 659.
Falle, S. A. E. G., 2002, ApJ, 557, L123.
Falle, S. A. E. G., Komissarov, S. S., & Joarder, P., 1998, MNRAS, 297, 265.
Fryxell, B., Olson, K., Ricker, P., Timmes, F. X., Zingale, M., Lamb, D. Q., MacNiece, P., Rosner, R., Truran, J. W., & Tofu, H., 2000, ApJS, 131, 273.
Gardiner, T. A. & Stone, J. M., 2005, J. Comput. Phys., 205, 509.
Hawley, J. F., & Stone, J. M., 1995, Comp. Phys. Comm., 89, 127.
Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P., 1992, *Numerical Recipes*, (Cambridge: Cambridge University Press).
Raga, A. C., Mellema, G., & Lundqvist, P., 1997, ApJS, 109, 517.
Ryu, D. & Jones, T. W., 1995, ApJ, 442, 228.
Stone, J. M., & Norman, M. L., 1992a, ApJS, 80, 759.
——., 1992b, ApJS, 80, 791.
Turner, N. J. & Stone, J. M., 2001, ApJS, 135, 95.
van Leer, B., 1977, J. Comput. Phys., 23, 276.
von Neumann, J., & Richtmyer, R. D., 1950, J. Appl. Phys., 21, 232.